

중간고사 – 실기시험

심해 탐험대 게임 제작

Unity를 이용한 2D 게임프로그래밍

동명대학교 게임공학과 | 강영민

2026년 1학기

시험 안내

시험 시간	120분 (2시간)
총점	100점
시험 형태	Unity 프로젝트 실기 제작
제출물	완성된 Unity 프로젝트에서 Script만 추출하여 제출(.zip)
허용 자료	강의 슬라이드, Unity 공식 문서, 인터넷 검색, AI 도구
사용 도구	Unity, Visual Studio IDE

유의사항

- 시험 시작 전 새 Unity 2D 프로젝트를 생성하고, 프로젝트 이름을 “MidTerm_학번_이름”으로 설정하십시오.
- 부분 점수를 부여하므로, 완성하지 못한 과제도 작성한 코드를 제출하십시오.
- 인터넷 검색과 AI 도구 사용이 허용됩니다. 단, 일부 과제는 설계, 자유 구현을 요구하므로 단순 복사·붙여넣기로는 해결되지 않습니다.
- 타인과 실시간 협업(화면 공유, 메신저 등)은 금지합니다.

☰ 배점 구성

전체 배점표

과제	주제	핵심 내용	배점
1	잠수함 조종	상하 이동, 조종 스크립트 구현, 경계 제한, Input System	25점
2	해류 생물 시스템	우→좌 해류 경로, ScriptableObject, 스폰, 난이도 설계	25점
3	전투 시스템	발사, 충돌, 컴포넌트 기반 구분, 점수 관리	35점
4	연출 효과	자유 이펙트 설계, 수중 충격파(카메라 셰이크)	15점
합계			100점

💡 권장 진행 순서

1. **과제 1** (약 20분): 잠수함 오브젝트를 만들고, 조종 스크립트를 구현하여 상하 이동과 경계 제한을 완성한다.
2. **과제 2** (약 30분): 우→좌 해류 경로, 프리팹, Wave 스폰, **난이도 증가 시스템을 설계**한다.
3. **과제 3** (약 35분): 발사, 컴포넌트 분기, 점수 관리를 구현한다.
4. **과제 4** (약 15분): **자유 이펙트를 설계**하고, 카메라 셰이크를 추가한다.
5. 나머지 시간: 테스트 및 버그 수정

1 게임 개요

🎮 “심해 탐험대 (Deep Sea Explorer)” 게임 설명

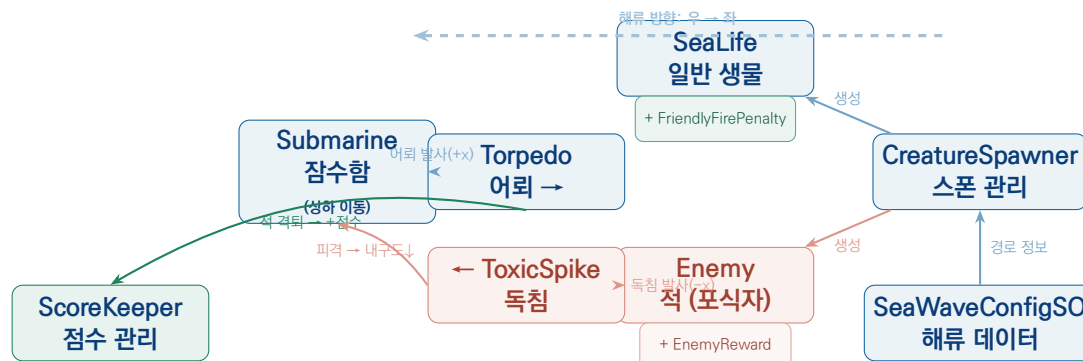
깊은 바닷속에서 **잠수함**을 상하로 조종하며, 화면 우측에서 좌측으로 흘러오는 생물들 사이에 숨은 **적(포식자)**을 식별하여 격퇴하는 **수중 사격 판별 게임**을 제작한다.

핵심 규칙:

- 해류를 따라 **일반 생물**(해파리, 거북이)과 **적(아귀, 독해파리)**이 섞여서 **우측에서 좌측으로** 이동해 온다.
- **적을 어뢰로 맞추면** → **점수 획득** (+10점~+30점)
- **일반 생물을 어뢰로 맞추면** → **잠수함 내구도 감소** (오사격 패널티)
- **적은 잠수함을 향해 독침을 발사**하여 공격한다 → 잠수함 내구도 감소
- 잠수함 내구도가 0이 되면 게임 오버

게임의 기본 흐름:

1. 플레이어는 키보드(W/S 또는 ↑/↓)로 잠수함을 **상하로만** 조종한다.
2. 화면 우측에서 해류 경로를 따라 일반 생물과 적이 섞여서 좌측으로 이동해 온다.
3. 플레이어는 적만 골라서 어뢰를 **우측(+x)** 방향으로 발사해야 한다 (판별 필요!).
4. 적은 자동으로 잠수함 방향(**좌측, -x**)으로 독침을 발사한다.
5. 적이 격퇴되면 거품 이펙트가 재생되고 점수가 증가한다.



2 사전 준비

시험 시작 후 아래 사전 준비를 먼저 완료하시오.

🔧 프로젝트 초기 설정

1. Unity Hub에서 **2D (URP)** 템플릿으로 새 프로젝트를 생성한다.
2. 프로젝트 이름: **MidTerm_학번_이름**
3. **Assets** 폴더 아래에 다음 하위 폴더를 생성한다:
 - Scripts – 모든 C# 스크립트
 - Prefabs – 프리팹 (잠수함, 적, 어뢰, 독침)
 - ScriptableObjects – SO 에셋 (해류 Wave 설정)
 - Sprites – 스프라이트 이미지
 - VFX – 파티클 시스템 프리팹
4. **New Input System** 패키지를 설치한다:
Window > Package Manager > Input System > Install
5. 시험에 필요한 스프라이트는 직접 준비하거나 Unity 기본 도형을 사용한다.

3 과제 1: 잠수함 조종

25점

1-1. Submarine 오브젝트 구성

7점

과제 1-1 Submarine 게임오브젝트 생성

1. 빈 게임오브젝트 “Submarine”을 생성하고, submarine.png 스프라이트를 적용한다.
2. **PolygonCollider2D**를 추가하고 Is Trigger를 체크한다.
3. **Rigidbody2D**를 추가하고 Body Type을 **Kinematic**으로 설정한다.
4. **Player Input** 컴포넌트를 추가하고, 기본 Input Actions를 생성 및 연결한다.
5. Submarine을 화면 **좌측 중앙**에 배치한다 (우측을 향하도록).

1-2. Submarine.cs 조종 스크립트

12점

과제 1-2 잠수함 상하 이동 구현

아래 의사코드를 참고하여 잠수함 상하 이동을 구현하는 Submarine.cs를 작성하시오.

요구사항:

- cruiseSpeed에 비례하여 **프레임 독립적**으로 이동해야 한다.
- **상하(y축)**로만 이동해야 한다 (좌우 이동 불가).
- **New Input System**의 콜백 OnMove()로 입력을 받아야 한다.
- 위치는 매 프레임 **누적(+)**으로 변경해야 한다 (대입이 아님).

Submarine.cs - 의사코드

Submarine (MonoBehaviour)

속성:

cruiseSpeed (float, 기본값 6, Inspector 노출)

rawInput (Vector2)

매 프레임(Update)마다:

deltaY = rawInput의 y 성분 * cruiseSpeed * Time.deltaTime

현재 위치에 (0, deltaY, 0)을 더한다

입력 콜백 OnMove(value):

rawInput에 value를 Vector2로 읽어 저장한다

 힌트

InputValue.Get<Vector2>()로 2D 입력을 받되, y 성분만 사용한다.

transform.position += new Vector3(0, deltaY, 0)으로 위치를 누적 변경한다.

1-3. 수중 활동 경계 제한

6점

과제 1-3 Viewport 좌표를 이용한 경계 제한

잠수함이 화면 상하 밖으로 나가지 않도록 경계를 설정하시오.

- Camera.main.ViewportToWorldPoint()를 사용하여 **y축** 최소/최대 좌표를 계산한다.
- 약간의 패딩(padding)을 적용한다.
- Awake()에서 경계값을 초기화하고, Update()에서 Mathf.Clamp()로 **y 위치만** 제한한다.
- x 위치는 고정(잠수함은 좌우로 이동하지 않음).

4 과제 2: 해류 생물 시스템

25점

2-1. SeaWaveConfigSO 스크립터블 오브젝트

6점

과제 2-1 SeaWaveConfigSO 작성

해류 경로와 스폰 설정을 담은 **ScriptableObject**를 작성하시오.

- [CreateAssetMenu] 속성으로 에디터에서 쉽게 생성할 수 있게 한다.
- 필드: creaturePrefab, currentPathPrefab, driftSpeed, timeBetweenSpawns, spawnVariance
- GetCurrentPath(): 해류 Waypoint 리스트 반환
- GetRandomSpawnTime(): 랜덤 스폰 시간 반환

중요: 같은 SO 구조로 일반 생물 Wave와 적 Wave를 각각 생성한다.

일반 생물 Wave에는 jellyfish/turtle 프리팹을, 적 Wave에는 anglerfish/toxic_jelly 프리팹을 할당한다.

2-2. 프리팹 구성 - 일반 생물 vs 적

5점

과제 2-2 생물/적 프리팹 생성 및 컴포넌트 구분

일반 생물과 적 프리팹을 각각 만들고, 부착하는 **컴포넌트의 차이**로 구분하시오.

1. 일반 생물 프리팹 (jellyfish, turtle):

- CurrentFollower.cs, Health.cs를 부착
- FriendlyFirePenalty.cs를 부착 (오사격 판별용)
- Launcher는 부착하지 **않는다** (공격 안 함)
- Collider2D (Trigger) + Rigidbody2D (Kinematic)

2. 적 프리팹 (anglerfish, toxic_jelly):

- CurrentFollower.cs, Health.cs를 부착
- EnemyReward.cs를 부착 (점수 보상용)
- Launcher.cs를 부착, autoAttack = true (독침 자동 발사)
- 독침은 좌측 방향(-x)으로 발사

	Follower	Health	Launcher	FriendlyFire	EnemyReward
해파리/거북이	O	O	X	O	X
아귀/독해파리	O	O	O	X	O

2-3. 해류 경로 및 Wave 구성

4점

과제 2-3 경로 + Wave SO 에셋 생성

1. 해류 경로를 최소 2개 만든다 (Current 0, Current 1), 각 4개 이상 Point.
경로는 **화면 우측 밖에서 시작하여 좌측으로 이동**하도록 구성한다.
2. SeaWaveConfigSO 에셋을 최소 3개 생성한다:
 - Wave 1: 해파리 (SeaLife) - Current 0 경로
 - Wave 2: 아귀 (Enemy) - Current 1 경로
 - Wave 3: 거북이 + 독해파리 혼합 - Current 0 경로
3. 일반 생물과 적이 **같은 해류에 섞여서** 나타나도록 Wave 순서를 구성한다.

2-4. CreatureSpawner.cs + CurrentFollower.cs

5점

과제 2-4 스폰 및 경로 이동

- CreatureSpawner.cs: Wave 리스트를 순회하며 코루틴으로 생물/적을 스폰한다. 생성된 오브젝트에 CurrentFollower.SetWaveConfig()로 해류 경로를 할당한다. 모든 Wave 완료 후 무한 반복한다.
- CurrentFollower.cs: Vector2.MoveTowards()로 Waypoint를 순차 이동하고, 마지막 Point 도달 시 Destroy()한다.

2-5. 🧩 난이도 증가 시스템 설계

5점

과제 2-5 🧩 설계 과제: 난이도 증가 시스템

Wave가 한 사이클 반복될 때마다 게임이 점점 어려워지도록 **난이도 증가 로직**을 CreatureSpawner.cs에 직접 설계하여 구현하시오.

조건:

- 최소 **2가지 이상의 파라미터**가 사이클마다 변화해야 한다.
예: 이동 속도, 스폰 간격, 적 비율, 내구도 등 - 어떤 조합이든 자유
- 난이도가 **무한히 증가하지 않도록** 상한(cap)을 설정해야 한다.
- 코드에 // 난이도 증가: 주석으로 어떤 파라미터를 왜 변경했는지 간략히 설명하시오.

※ 정답이 하나가 아닌 **설계 과제**입니다. 구현의 완성도와 게임플레이 타당성을 평가합니다.

5 과제 3: 전투 시스템

35점

이 과제가 게임의 핵심이다. 적과 생물을 **컴포넌트로 구분**하여 적중 시 다른 결과를 처리해야 한다.

3-1. Launcher.cs - 어뢰/독침 발사

9점

과제 3-1 발사 시스템 구현

참수함과 적 모두에게 사용할 수 있는 범용 Launcher.cs를 작성하시오.

- 필드: projectilePrefab, projectileSpeed, launchRate, bool autoAttack
- 함수: OnFire(InputValue value) 콜백으로 수동 발사 (**우측, +x**)
- 적: autoAttack = true이면 자동 발사 (**좌측, -x**)
- Coroutine으로 launchRate 간격 연속 발사
- Rigidbody2D.linearVelocity = transform.right * projectileSpeed

Launcher.cs - 의사코드

Launcher (MonoBehaviour)

속성:

projectilePrefab (발사체 프리팹, Inspector 노출)

projectileSpeed (발사 속도, Inspector 노출)

launchRate (발사 간격, Inspector 노출)

autoAttack (자동 발사 여부, Inspector 노출)

코루틴 -- 연속 발사:

무한 반복:

현재 위치에 projectilePrefab을 복제 생성한다

생성된 발사체의 Rigidbody2D 속도를

자신의 오른쪽 방향 × projectileSpeed로 설정한다

launchRate 초만큼 대기한다

3-2. 레이어 설정 및 충돌 매트릭스

5점

과제 3-2 레이어 및 충돌 구성

오브젝트 간 올바른 충돌이 발생하도록 **4개 레이어**를 설정하시오.
일반 생물과 적은 같은 Creature 레이어를 사용한다 (구분은 컴포넌트로 처리).

- 레이어: Submarine, Creature, Torpedo, ToxicSpike
- 충돌 매트릭스:

	Submarine	Creature	Torpedo	ToxicSpike
Submarine	-	-	-	0
Creature		-	0	-
Torpedo			-	-
ToxicSpike				-

- 어뢰(Torpedo)는 Creature와 충돌 → Health.cs에서 **컴포넌트 유무**로 결과 구분
- 독침(ToxicSpike)은 Submarine과만 충돌 → 잠수함 내구도 감소
- Submarine과 Creature는 직접 충돌하지 않음 (발사체를 통해서만 상호작용)

3-3. Health.cs - 내구도 관리

5점

과제 3-3 내구도 시스템 구현

모든 파괴 가능 오브젝트에 부착하는 Health.cs를 작성하시오.

- [SerializeField] int hitPoints - 내구도 (잠수함: 10, 적: 1~3, 생물: 1)
- OnTriggerEnter2D()에서 DamageDealer를 가져와 내구도를 감소시킨다.
- 내구도가 0 이하가 되면 Destroy(gameObject)한다.
- public void TakeDamage(int amount) 메서드를 만들어 외부에서도 내구도를 감소시킬 수 있게 한다.
- 발사체는 DamageDealer.Hit()으로 제거한다.

3-4. 오사격 판별 - FriendlyFirePenalty.cs + EnemyReward.cs

8점

과제 3-4 핵심: 컴포넌트 기반 적/생물 구분

파괴 시 결과를 구분하기 위해 **별도 컴포넌트**를 사용한다. GetComponent<T>()로 컴포넌트 유무를 확인하여 분기하는 것이 핵심이다.

(A) FriendlyFirePenalty.cs - 일반 생물 프리팹에만 부착:

- [SerializeField] int penaltyAmount = 1 - 오사격 시 잠수함에 가할 피해량
- public int GetPenalty() 메서드로 페널티 값을 반환
- public void ApplyPenalty() 메서드:
 - FindAnyObjectByType<Submarine>()로 잠수함을 찾는다
 - 잠수함의 Health.TakeDamage(penaltyAmount)를 호출한다

(B) EnemyReward.cs - 적 프리팹에만 부착:

- [SerializeField] int pointValue = 10 - 격퇴 시 획득 점수
- public int GetPoints() 메서드로 점수를 반환
- public void GrantReward() 메서드:
 - FindAnyObjectByType<ScoreKeeper>()로 점수 관리자를 찾는다
 - ScoreKeeper.AddScore(pointValue)를 호출한다
 - 거품 이펙트 재생 + 카메라 셰이크 호출

(C) Health.cs의 파괴 처리에서 GetComponent<>()로 분기:

Health.cs - 파괴 시 컴포넌트 기반 분기 (핵심 로직 의사코드)

파괴 처리:

자신에게서 EnemyReward 컴포넌트를 가져온다

만약 EnemyReward가 있으면:

보상을 지급한다 (점수 추가)

자신에게서 FriendlyFirePenalty 컴포넌트를 가져온다

만약 FriendlyFirePenalty가 있으면:

페널티를 적용한다 (잠수함 내구도 감소)

자신을 파괴한다

💡 힌트: GetComponent 패턴

GetComponent<T>()는 해당 컴포넌트가 없으면 null을 반환한다.

이 특성을 이용하면 **컴포넌트의 유무 자체**가 오브젝트의 종류를 판별하는 수단이 된다.

이 패턴은 강의에서 DamageDealer를 GetComponent<>()로 가져오는 것과 동일하다.

3-5. DamageDealer.cs

3점

과제 3-5 피해 컴포넌트

어뢰/독침에 부착하는 DamageDealer.cs를 작성하시오. GetDamage(), Hit() 메서드를 구현한다.

3-6. ScoreKeeper.cs – 점수 관리

5점

과제 3-6 점수 시스템 구현 – 기본

적 격퇴 시 점수를 관리하고 화면에 표시하는 ScoreKeeper.cs를 작성하시오.

- 빈 게임오브젝트 “ScoreKeeper”를 만들고 스크립트를 부착한다.
- `int score` 변수로 현재 점수를 관리한다.
- `[SerializeField] TextMeshProUGUI scoreText` – UI 텍스트 연결
- `public void AddScore(int points)`: 점수를 더하고 UI를 갱신한다.
- `public int GetScore()`: 현재 점수를 반환한다.
- Canvas에 TextMeshPro 텍스트를 배치하여 점수를 표시한다.

6 과제 4: 연출 효과

15점

4-1. 🎮 자유 연출 설계: 격퇴 이펙트

9점

과제 4-1 적 격퇴 시 시각 연출 - 자유 설계

적(Enemy)이 격퇴될 때 재생할 **시각 이펙트**를 **직접 설계**하여 구현하시오.

필수 조건:

- **Particle System**을 사용해야 한다 (최소 1개).
- Looping이 아닌 **1회성** 이펙트여야 한다.
- 프리팹으로 저장하고, Health.cs에서 적 파괴 시 Instantiate()로 생성, 재생 완료 후 Destroy()해야 한다.

자유 요소 (아래 항목을 직접 결정하시오):

- 이펙트의 컨셉 (거품 폭발, 먹물 분사, 전기 방전, 소용돌이 등)
- Shape, Color, Size, Speed 등 Particle System 파라미터
- Duration 및 파티클 수

채점 기준: 기술적 완성도(프리팹화, 자동 파괴) + **연출의 심해 테마 적합성 및 독창성**

※ Inspector 캡처 또는 코드 주석으로 설계 의도를 간략히 설명하면 가산점

4-2. 수중 충격파 (카메라 셰이크)

6점

과제 4-2 카메라 흔들림 효과

적 격퇴 시 화면이 흔들리는 ShakeCam.cs를 구현하시오.

- Main Camera에 부착
- Shake(float duration, float magnitude) 코루틴
- Random.insideUnitCircle * magnitude로 오프셋 계산
- duration 종료 후 원위치 복귀
- Health.cs에서 적(Enemy) 파괴 시 호출
- **추가 도전:** 적의 종류에 따라 흔들림 강도를 다르게 설정하면 가산점

7 채점 기준표

☑ 상세 채점 기준

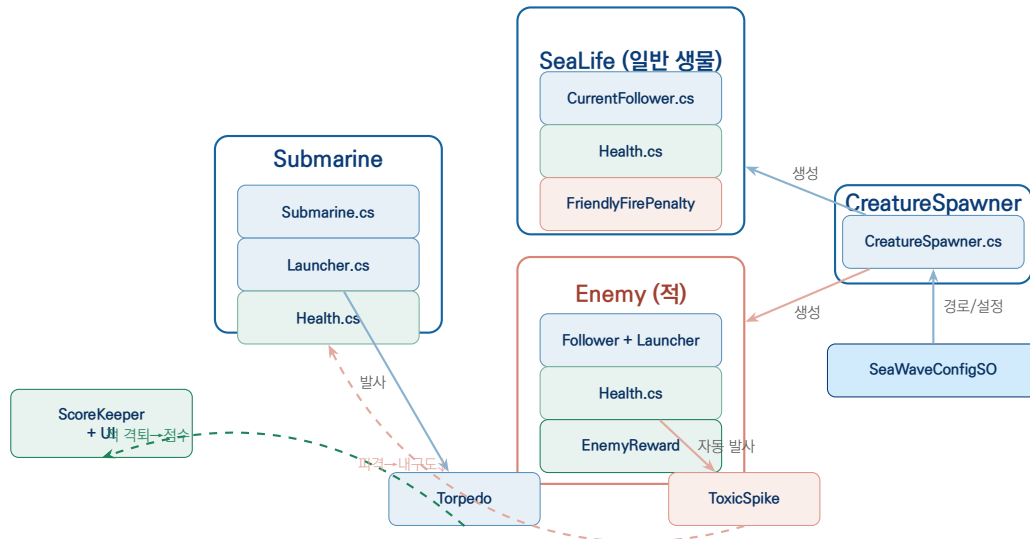
과제	항목	채점 기준	배점
1	오브젝트 구성	Collider, Rigidbody, Input 컴포넌트 설정	7
	조종 스크립트	의사코드 기반 구현, 프레임 독립 이동, y축 전용	12
	경계 제한	ViewportToWorldPoint 사용, y축 Clamp 적용	6
2	SeaWaveConfigSO	SO 구조, GetCurrentPath, GetRandomSpawnTime	6
	프리팸 구성	생물/적 프리팸 컴포넌트 차이 구성	5
	해류 경로/Wave	최소 2경로, 3개 이상 Wave, 생물/적 혼합	4
	Spawner/Follower	코루틴 스폰, Waypoint 이동, Wave 반복	5
	난이도 설계 🎮	2개+ 파라미터 변화, 상한 설정, 타당성	5
3	Launcher	수동/자동 발사, 코루틴, 방향 설정	9
	레이어 설정	4개 레이어, 충돌 매트릭스 올바르게 구성	5
	Health	내구도 감소, TakeDamage, 파괴 처리	5
	컴포넌트 분기	EnemyReward→점수, FriendlyFire→페널티	8
	DamageDealer	GetDamage, Hit, 발사체 제거	3
	ScoreKeeper	점수 관리, TextMeshPro UI 표시, AddScore	5
4	자유 이펙트 🎮	Particle System, 프리팸화, 테마 적합성, 독창성	9
	수중 충격파	코루틴, Random 오프셋, 원위치 복귀	6
합계			100

★ 보너스 점수 (최대 +10점)

- +3점: 잠수함 내구도를 UI(슬라이더 또는 산소 게이지)로 표시
- +3점: 심해 배경 스크롤링 효과 구현
- +2점: 오사격 시 화면 가장자리가 빨갛게 깜빡이는 경고 연출
- +2점: 적 종류별 차등 점수 (아귀: 30점, 독해파리: 10점)

※ 보너스 점수를 합산해도 총점은 100점을 초과하지 않음

8 참고: 전체 아키텍처



스크립트 간 관계 요약

스크립트	역할 및 연결
Submarine.cs	Input System으로 상하 조종 입력 처리, y축 경계 제한
CurrentFollower.cs	SeaWaveConfigSO에서 해류 Waypoint를 받아 경로 이동 (생물/적 공용)
SeaWaveConfigSO	프리팸, 해류 경로, 스폰 간격 등 Wave 데이터 (ScriptableObject)
CreatureSpawner.cs	Wave 리스트를 기반으로 생물/적을 코루틴으로 순차 생성
Launcher.cs	수동 어뢰(+x)/자동 독침(-x) 발사, 코루틴 연속 발사
DamageDealer.cs	발사체에 부착, 피해량 제공 및 충돌 후 자기 파괴
Health.cs	내구도 관리, 파괴 시 GetComponent로 분기 처리 호출
EnemyReward.cs	적 전용 - 격퇴 시 ScoreKeeper에 점수 추가 + 이펙트 재생
FriendlyFirePenalty.cs	생물 전용 - 오사격 시 함수 Health.TakeDamage() 호출
ScoreKeeper.cs	점수 관리, TextMeshPro로 점수 표시
ShakeCam.cs	코루틴 카메라 흔들림, 적 격퇴 시 호출

 이 시험의 핵심 포인트

이 게임은 강의의 아케이드 게임과 달리 “아무거나 쏘면 안 되는” 판별 메카닉이 핵심이다.

- 일반 생물에는 FriendlyFirePenalty를, 적에는 EnemyReward를 부착한다.
- Health.cs의 파괴 처리에서 GetComponent<>()로 컴포넌트 유무를 확인하여 분기한다.
- EnemyReward 있음 → 적 격퇴 → 점수 보상
- FriendlyFirePenalty 있음 → 생물 오사격 → 내구도 감소
- 이 **컴포넌트 기반 구분** 로직이 정확히 동작해야 게임이 성립한다.

 설계 과제(총 2개)에 대하여:

- 난이도 증가, 자유 이펙트 - 정답이 하나가 아닌 **오픈엔드 과제**
- 기술적 완성도 + **설계의 타당성과 독창성**을 종합 평가
- 코드 주석으로 설계 의도를 설명하면 가산점

— 시험 끝 —